

Operators

→ Symbols that tells compiler to perform some operation.
<operand> <operator> <operand>

binary operator → 2 operand.

Arithmetic Operator.

Binary → +, -, *, /, %

Unary → ++, --
(Increment) (Decrement)

$a = a + 1 \rightarrow a++$ / $f + a$

$a = a - 1 \rightarrow a--$ / $--a$

Pre Increment

$[++a]$ ① change value
② use value.

int a = 10;

int b = 0;

b = ++a;

System.out.println(a);

System.out.println(b);

↓

b = 11, a = 11

Post Increment

$[a++]$ ① use value
② change value.

int a = 10;

int b = 0;

b = a++;

System.out.println(a);

System.out.println(b);

↓

b = 10, a = 11

Pre-decrement

$[--a]$

int a = 10;

int b = 0;

b = --a;

System.out.println(a);

System.out.println(b);

↓

b = 9, a = 9

Post Decrement

$[a--]$

int a = 10;

int b = 0;

b = a--;

System.out.println(a);

System.out.println(b);

↓

b = 10, a = 9

Relational Operators $\rightarrow$ output in boolean
 $=$, $!=$, $>$, $<$, $>=$, $<=$

Logical operators $\rightarrow$ boolean
 $||$, $||$, $!$

1	1	1
0	0	0
0	1	1
1	0	0

Bitwise Operators $\rightarrow$ output 1 or 0
 $&$, $|$, $\wedge$, $<<$, $>>$, $\sim$ $\rightarrow$ binary complement

Assignment Operators
 $=$, $+=$, $-=$, $*=$, $/=$

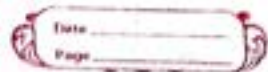
Conditional / Ternary Operator
expression 1 ? expression 2 : expression 3

Misc Operators

- $\rightarrow$ Java Member Access Operator ($.$)
- $\rightarrow$ Java Comma Operator ($,$)
- $\rightarrow$ Java Array Index Operator ($[]$)
- $\rightarrow$ Java new Operator.

Smalltalk → 100% OOP programming language

OOPs



- OOP is a programming paradigm / methodology
- Paradigm can also be termed as method to solve some problem or do some task. Programming paradigm is an approach to solve problem using some programming language or also we can say it is a method to solve a problem using tools and techniques that are available to us following some approach.

→ Programming Paradigm

↳ Object Oriented paradigm ✓

↳ Procedural paradigm

↳ Functional paradigm

GFCS

↳ Logical paradigm

↳ Structured paradigm

→ 6 main pillars OOPs are:

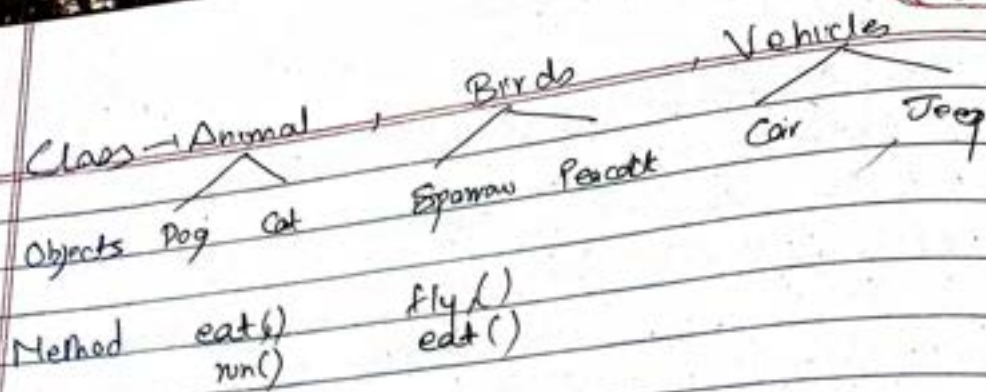
- (i) Class
- (ii) Object & methods
- (iii) Inheritance
- (iv) Polymorphism
- (v) Abstraction
- (vi) Encapsulation

- Platform independent
- Object Oriented
- Simple & Easy to learn
- Multithreading
- Secure

WORA

Components of Java

- Access
- Modifiers
- Superclass
- Interface
- Keyword
- Class Name
- Body



- Class is the collection of objects
- Class is not a real world entity. It is just a template or blueprint.
- Class does not occupy memory.

Syntax:

~~class~~ access-modifier class ClassName.

- {
- methods
- Constructors
- fields
- block
- nested class
- }

;

- Method is a set of codes which performs a particular task.
- Advantages of method
 - Code Reusability
 - Code Optimization

Syntax:

access-modifier return-type methodName (List of parameters)

{

}

- Date _____
Page _____
- Object is an instance of class
 - Object is real world entity.
 - Object occupies memory.
 - Object consists of
 - 1) Identity → Name of dog
 - 2) State / Attribute → Breed, Age, Colour.
 - 3) Behaviour → Eat, Run, Bark.

How to create an object:

- (i) By using new keyword.
- (ii) By using newInstance() method
- (iii) clone() method
- (iv) deserialization.
- (v) By using factory methods.

→ Ways to create object

- (i) Declaration `Animal miri;`
- (ii) Instantiation `miri = new Animal();`
- (iii) Initialization. `miri = new Animal();`

Syntax: `Animal miri = new Animal();`

→ Method call:

`miri.run();`
`miri.eat();`

Page _____

```
public class Animal
{
    public static void eat()
    {
        System.out.println("I am eating");
    }
}
```

```
public static void main (String[] args)
{
    Animal mini = new Animal();
    mini.eat();
}
```

```
public class Birds
{
```

```
    public static void fly()
    {
        System.out.println("I am flying");
    }
}
```

```
public static void main (String[] args)
{
```

```
    Birds s1 = new Birds();
    s1.fly();
}
```


Methods to initialize objects

(i) By reference variable.

```
public class Animal
{
    String color;
    int age;

    public static void main (String[] args)
    {
        Animal ab = new Animal();
        ab.color = "black";
        ab.age = 15;

        System.out.println("Color is " + ab.color);
        System.out.println("Age is " + ab.age);
    }
}
```

3.

(ii) By using method.

```
public class Animal
{
    String color;
    int age;

    void initObj (String c, int a)
    {
        color = c;
        age = a;
    }

    void display()
    {
        System.out.println(color + " " + age);
    }

    public static void main (String[] args)
    {
        Animal ab = new Animal();
        ab.initObj("black", 10);
        ab.display();
    }
}
```


③ Using Constructor

- doesn't have return type.
- Constructor is a block (similar to method) having same name as of class name.

```
public class Test
{
    public void test()
    {
        // ...
    }
}
```

S X not constructor

- The only modifiers applicable for constructor are public, protected, default & private.

- It executes automatically when we create an object.

```
public class Test
{
```

```
    public Test()
    {
        // ...
    }
}
```

```
public static void main (String[] args)
{
```

```
    Test t = new Test();
}
```

```
}
```


e1

String
emp_id

String
emp_id

```
public class Employee
```

```
    String name;  
    int emp_id;
```

```
    public Employee (String name, int emp_id)  
    {  
        this.name = name;  
        this.emp_id = emp_id;  
    }
```

```
    public static void main (String[] args)
```

```
    {  
        Employee e1 = new Employee ("deepak", 101);  
        Employee e2 = new Employee ("sunu", 102);  
    }
```

Uses of objects constructor

- (I) To initialize an object
- (II) To create an object X

↓
new

Date _____
Page _____

Types of constructor

- ① Default constructor (No-Argument constructor)
- ② No-Argument constructor (User-defined)
- ③ Parametrized constructor.

① Default Constructor.
↳ created by compiler:

```
public class test
{
    public test()
    {
        super();
    }
    public static void main(String[] args)
    {
        test t = new test();
    }
}
```

② No-Argument Constructor.

```
public class test
{
    public test()
    {
    }
}

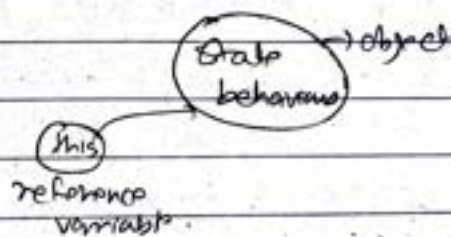
public static void main(String[] args)
{
    test t = new test();
}
}
```


(ii) Parameterized Constructor.

```
public class test
{
    public test (String name)
    {
        this.name = name;
    }

    public static void main (String[] args)
    {
        test t = new test ("sunu");
    }
}
```

This - Keyword

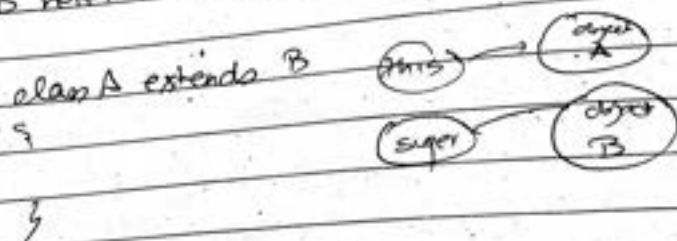


- refers to the current class instance variable.
- can be used to invoke current class method (implicitly).
- can be used to invoke current class constructor.
- can be used to pass an argument in the method call.
- can be used to pass as an argument in constructor call.
- can be used to return the current class instance from the method.

Date _____
Page _____

Super keyword

→ Super keyword is a reference variable which is used to refer immediate parent class object



```

class A
{
    int a=10;
}

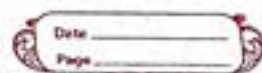
class B extends A
{
    int a=20;
    void show (int a)
    {
        System.out.println(a); // 30
        // sop (this.a) // 20
        // sop (super.a) // 10
    }
}

public static void main ( )
{
    B obj = new B();
    obj.show(30);
}
  
```

- "Super" keyword can be used to refer immediate parent class instance variable.
- "super" keyword can be used to invoke immediate parent class method.
- super() can be used to invoke immediate parent class constructor.

Final Keyword

use with
variable
method
→ class.



- If we create any final variable, it becomes constant, we cannot change the value of final variable.
- If we create any final method, we cannot override it.
- If we create any final class, we cannot extend it or inherit it.

```
class Test
{
    public static void main()
    {
        final int i = 10;
        i = i + 20; // error
        sop(i);
    }
}
```

```
class Demo
{
    final void m1()
    {
        sop("I am dancing");
    }
}
```

```
class Test extends Demo
{
    void m1() // error
    {
        sop("I am singing");
    }
    psvm()
    {
    }
}
```

```
final class Demo
{
    void m1()
    {
        sop("I am dancing");
    }
}
```

```
class Test extends Demo // error
{
    void m1()
    {
    }
    psvm()
    {
    }
}
```


Static

Access Modifiers → public, private, protected, default.
Non-access modifier → static, final, abstract, synchronized, transient, volatile, strictfp

- Used with
- (i) Class level variable (X local variable)
 - (ii) Methods
 - (iii) block
 - (iv) Inner class (X outer class)
(nested)

class Test

```
{ static int a = 10;
```

```
}
```

class Demo

```
{ psvm()
```

```
{ System.out.println(Test.a)
```

```
}
```

```
}
```

class Employee;

```
{ int empId;
```

```
String name;
```

```
String company;
```

Employee (int an

void display()

```
{ sop(empId + " " + name + " " + company);
```

```
}
```

```
psvm()
```

```
{ Employee e1 = new Employee(101, "amit", "SP");
```


class Employee

{

int empId;

String name;

static String company = "SP";

Employee (int empId, String name, String company)

{ this.empId = empId;

this.name = name;

// this.company = company;

}

{

void display()

{ System.out.println(empId + " " + name + " " + company)

}

public static void main (String[] args)

{

Employee e1 = new Employee (101, "amit", "SP");

e1.display();

Employee e2 = new Employee (102, "deepak", "SP");

e2.display();

}

}

Q1

Q2

Q3

Program Examples

Date _____
Page _____

Page no. _____

```
public class Student
```

```
{  
    public static class Student
```

```
    {  
        String name;
```

```
        int roll;
```

```
        double percentage;
```

```
    }
```

```
    public static void main (String[] args)
```

```
{
```

```
    Student s1 = new Student();
```

```
    s1.name = "Sunu";
```

```
    s1.roll = 12;
```

```
    s1.percentage = 95.9;
```

```
    System.out.println(s1.name);
```

```
    System.out.println(s1.roll);
```

```
    System.out.println(s1.percentage);
```

```
}
```

```
}
```


public class Student

{
 public static class Student
 {
 String name;
 int roll;
 double percentage;
 }
}

public static void main (String[] args)

{
 Student s1 = new Student();
 s1.name = "Bunu";
 s1.roll = 12;
 s1.percentage = 95.5;

Student s2 = new Student();
 s2.name = "Jhope";
 s2.roll = 123;
 s2.percentage = 92.5;

System.out.println("Name of s1: " + s1.name);
System.out.println("Name of s2: " + s2.name);
}

Date _____
Page _____

```
public class Student Car
```

```
public static class Student
```

```
{ String name;
```

```
int roll;
```

```
double percentage;
```

```
}
```

```
public static class Car
```

```
{ String name;
```

```
String type;
```

```
int price;
```

```
}
```

```
public static void main (String[] args)
```

```
{
```

```
Student ram = new Student();
```

```
ram.name = "Ram";
```

```
ram.roll = 123;
```

```
ram.percentage = 95.7;
```

```
Car c1 = new Car();
```

```
c1.name = "Ford";
```

```
c1.type = "Sports Coupe";
```

```
c1.price = 105000;
```

```
System.out.println ("Name of Student is " + ram.name);
```

```
System.out.println ("Car he owns: " + c1.name);
```

```
}
```

```
}
```


by method



```
public class StudentClass
```

```
{  
    public static class Student  
    {  
        String name;  
        int roll;  
        double percentage;  
    }  
}
```

```
public void fun (Student x).  
{  
    System.out.println (x.name);  
}
```

```
public static void main (String[] args)
```

```
{  
    Student x = new Student();  
    x.name = "John";  
    x.roll = 123;  
    x.percentage = 3.14;  
}
```

```
Student.fun (x);  
}
```


Getter & Setter.



```
public class StudentClass
{
    public static class Student
    {
        String name;
        private int roll;
        double percentage;

        public int getRoll()
        {
            return roll;
        }
    }
}
```

```
public class main
{
    public static void main(String [] args)
```

```
    {
        StudentClass.Student s1 = new StudentClass.Student();
        s1.name = "Suvu";
        //s1.roll = 76;
        s1.percentage = 3.14;
        System.out.println(s1.getRoll());
    }
}
```

output-10

public class StudentClan
{
 public static class Student

String name;
private int roll;
double percentage;

public int getRoll()
{
 return roll;
}

public void setRoll(int roll)
{
 this.roll = roll;
}

}

public class main

{
 public static void main(String[] args)

{
 Student^{clan} Student s1 = new StudentClan.Student();

s1.name = "Samu";

s1.setRoll(26);

s1.percentage = 3.14;

System.out.println("Name = " + s1.name);

System.out.println("Roll = " + s1.getRoll());

System.out.println("percent = " + s1.percentage());

}

}

Example

```
public class StudentClass  
{  
    public static class Student
```

```
        String name;  
        int roll;  
        double percentage;
```

} instance variable
local variable

```
    public Student (String naam, int r-no, double per)  
    {  
        name = naam;  
        roll = r-no;  
        percentage = per;  
    }  
}
```

```
public class main
```

```
{  
    public static void main (String[] args)
```

```
    {  
        Student s1 = new Student ("sunu", 46, 99.7);  
        Student s2 = new Student ("ram", 48, 99.8);
```

```
        System.out.println (s1.name + " " + s2.name);  
        System.out.println (s1.roll + " " + s2.roll);  
        System.out.println (s1.per + " " + s2.per);  
    }  
}
```


public class Employee

{

private String name;

private int ~~not~~ id;

private double salary;

public Employee(String name, int id, double salary)

{

this.name = name;

this.id = id;

this.salary = salary;

}

}

public class main

public static void main(String[] args)

{

Employee e1 = new Employee("Sonu", 123, 15400);

System.out.println(e1.name);

System.out.println(e1.id);

System.out.println(e1.salary);

}

}

Bank

```
public class bank
```

```
{  
    String name;  
    String address;  
    double balance;  
}
```

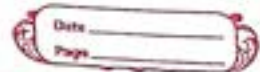
```
public bank (String name, String address, double balance)  
{  
    this.name = name;  
    this.address = address;  
    this.balance = balance;  
}
```

```
public class main
```

```
{  
    public static void main (String[] args)
```

```
{  
        bank s1 = new bank ("mukhnath", "kamladi", 29324.5)  
        System.out.println (s1.name);  
        System.out.println (s1.address);  
        System.out.println (s1.balance);  
    }  
}
```


Laptop



```
public class Laptop {  
    private String brand;  
    private String name;  
    private String laptopcode;  
    private String color;  
    private int ram;  
    private boolean hartschscreen;
```

```
    public Laptop(String brand, String name, String laptopcode,  
        String color, int ram, boolean hartschscreen) {  
        this.brand = brand;  
        this.name = name;  
        this.laptopcode = laptopcode;  
        this.color = color;  
        this.ram = ram;  
        this.hartschscreen = hartschscreen;  
    }
```

```
    public String toString() {  
        return ("Brand: " + this.brand + "\n" + "name: " + this.name +  
            "\n" + "laptopcode: " + this.laptopcode + "\n" +  
            "color: " + this.color + "\n" + "ram: " + this.ram + "\n" +  
            "has hartschscreen: " + this.hartschscreen);  
    }
```

```
public class main {  
    public static void main (String[] args) {  
        Laptop Dell = new Laptop("Dell", "vostro", "ABC", "black", 16, false);  
        System.out.println(Dell.toString());  
    }
```


Final

```
public class StudentClass
{
    public static class Student
```

```
    {
        String name;
        int roll;
        double percentage;
        final String SchoolName = "RIBS";
```

```
    public Student (String name, int roll, double percentage)
    {
        this.name = name;
        this.roll = roll;
        this.percentage = percentage;
    }
}
```

```
public class main
{
    public static void main (String [] args)
```

```
    {
        StudentClass.Student s2 = new StudentClass.Student
            ("sunu", 46, 32);
```

```
        System.out.println (s2.SchoolName);
    }
}
```


Static → memory management

Date
Page

```
public class StudentClass  
{  
    public static class Student
```

```
    {  
        String name;  
        int roll;  
        double percentage;  
        int numberOfStudents;
```

```
    }  
    public Student (String name, int roll, double percentage)  
    {  
        this.name = name;  
        this.roll = roll;  
        this.percentage = percentage;  
        numberOfStudents++;  
    }  
}
```

```
public class main  
{  
    public static void main (String[] args)
```

```
    {  
        StudentClass.Student s1 = new StudentClass.Student ("Suru", 96, 99.9);  
        StudentClass.Student s2 = new StudentClass.Student ("Rahul", 44, 91.7);
```

```
        System.out.println (s1.numberOfStudents);  
        System.out.println (s2.numberOfStudents);  
    }  
}
```

output
1
1

roll	name	percentage
96	Suru	99.9

roll	name	percentage
44	Rahul	91.7

Date _____
Page _____

```
public class StudentClass  
{  
    public static class Student
```

```
    {  
        String name;  
        int roll;  
        double percentage;  
        static int numberOfStudents;
```

```
    public Student(String name, int roll, double percentage)  
    {  
        this.name = name;  
        this.roll = roll;  
        this.percentage = percentage;  
        numberOfStudents++;  
    }  
}
```

```
public class main
```

```
{  
    public static void main(String[] args)
```

```
{  
        StudentClass.Student s1 = new StudentClass.Student("Ravi", 16, 99.5);  
        StudentClass.Student s2 = new StudentClass.Student("Rahul", 44, 92);
```

```
        System.out.println(s1.numberOfStudents);  
        System.out.println(s2.numberOfStudents);  
    }  
}
```

s1 [roll] [per]
s2 [roll] [per]
[numberOfStudents]

Output
1
2

Abstraction

```
abstract class Vehicle
{
    int no. of tyres;
    abstract void start();
}
```

```
class Car extends Vehicle
```

```
{
    void start()
    {
        System.out.println("Starts with key");
    }
}
```

```
class Scooter extends Vehicle
```

```
{
    void start()
    {
        System.out.println("Starts with kick");
    }
}
```

```
public class main
```

```
{
    public static void main (String[] args)
```

```
{
        Car c = new Car();
        c.start();
    }
```

```
    Scooter s = new Scooter();
```

```
    s.start();
}
```


abstract class Shape

{

abstract void draw();

}

class Circle extends Shape

{

void draw()

{ System.out.println("Drawing Circle");

}

}

class Rectangle extends Shape

{

void draw()

{ System.out.println("Drawing Rectangle");

}

}

public class main

{ public static void main(String[] args) {

Circle c = new Circle();

c.draw();

}

}

- belongs to class not object.
- can be called directly by class name as:
`ClassName.methodName();`

Static Methods

```

class Test
{
    void display()
    {
        sop("1");
    }

    static void show()
    {
        sop("2");
    }

    public static void main (String[] args)
    {
        Test t = new Test();
        t.display();
        Test.show();
    }
}

```

Rules

- Static methods belongs to the class, not to the object.
- Static methods can be accessed directly by class name and doesn't need any object.
- ① → Static method can access only static data. It cannot access non-static data (instance data).
- ② → Static method can call only other static methods and cannot call a non-static method.
- ③ → Static method cannot refer to "this" or "super" keyword.


```

(3) class StaticDemo
{
    static int i = 10;
    static void display()
    {
        System.out.println(i);
    }
}

```

```

(4) class StaticDemo
{
    static void display()
    {
        show();
        System.out.println("1");
    }

    static void show()
    {
        System.out.println("2");
    }
}

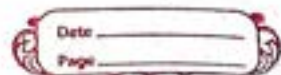
```

```

(5) class StaticDemo
{
    int i = 10;
    static void display()
    {
        System.out.println(this.i); // error
    }
}

```


Encapsulation



→ medicine capsule. (T)

→ Car engine

→ Encapsulation in Java is a mechanism of wrapping the data (variables) and code acting on the data (methods) together as a single unit

→ Steps to achieve encapsulation.

- 1) Declare the variables of a class as private.
- 2) Provide public setter and getter methods to modify and view the variables values.

Class Employee

```
1. private int emp-id; // data hiding
```

2.

→ In encapsulation, the variables of a class will be hidden from other classes and can be accessed only through the methods of their current class. This concept is known as data hiding

→ It helps to achieve security of data.

class Employee

{

① private int emp-id;

② public void setEmpId (int emp-id)

{

emp-id = emp-id;

}

③ public int getEmpId ()

{ return emp-id;

}

}

class Company

{ public static void main (String[] args)

{

Employee e = new Employee();

e.setEmpId(101);

System.out.println (e.getEmpId());

}

}

Polymorphism

Poly morphism

↓ many forms

→ Water
solid liquid gas

Shape
circle triangle

→ Sound
dog cat
bark meow

Types

Static → 1) Compile-Time polymorphism
dynamic → 2) Run-Time polymorphism

① Compile-Time Polymorphism

- Static polymorphism
- method overloading (achieve)
- handled by compiler

② Run-Time Polymorphism

- Dynamic Polymorphism
- method overriding (achieve)
- handled by JVM

Method overloading

- 1) same name
- 2) same class
- 3) Different arguments
 - No. of arg.
 - Sequence of arg.
 - Type of arg.

one.

Method overriding

- 1) Same name
- 2) Different class
- 3) Same arguments
 - No. of arg.
 - Sequence of arg.
 - Type of arg.
- 4) Inheritance (IS-A)

Method overloading (Compile - Time polymorphism)

```
class Test
```

```
{  
    void show()  
    {  
        System.out.println("1");  
    }  
}
```

```
void show()  
{  
    System.out.println("2");  
}
```

```
public static void main (String[] args)
```

```
{  
    Test t = new Test();  
    t.show();  
}
```

→ which show?

ambiguity error

// no. of argument diff

```
class Test  
{  
    void show()  
    {  
        System.out.println("1");  
    }  
}
```

```
void show(int a)  
{  
    System.out.println("2");  
}
```

```
public static void main(String[] args)  
{  
    Test t = new Test();  
    t.show(10);  
    t.show();  
}
```

→ 2
→ 1

3.

class Test

// sequence of arg diff.

```
1 void show (int a, String b)
2   System.out.println("1");
3
```

```
1 void show (String a, int b)
2   System.out.println("2");
3
```

```
1 public static void main (String[] args)
```

```
2   Test t = new Test();
```

```
3   t.show(10, "sunu"); → 1
```

```
4   t.show("sunu", 20); → 2
```

```
5
```

```
6
```


// type of arg diff

class Test

void show(int a)

System.out.println("1");

void show(String a)

System.out.println("2");

public static void main(String[] args)

Test t = new Test();

t.show(10); → 1

t.show("sunny"); → 2

Important Qns

Date _____
Page _____

- 1) Can we achieve method overloading by changing the return type of method only?
void show(int a) X
void show(int a) → no (ambiguity error).
- 2) Can we overload java main method?

class Test

```
1 public static void main (String[] args)
2 {
3     System.out.println("1");
4     Test t = new Test();
5     t.main(20);
6 }
```

```
1 public static void main (int a)
2 {
3     System.out.println("2");
4 }
```

→ output
1
2

3) class Test

```
1 void show (int a)
```

```
2 {
3     System.out.println("int method");
4 }
```

```
1 void show (String a)
```

```
2 {
3     System.out.println("String method");
4 }
```

```
1 public static void main (String[] args)
```

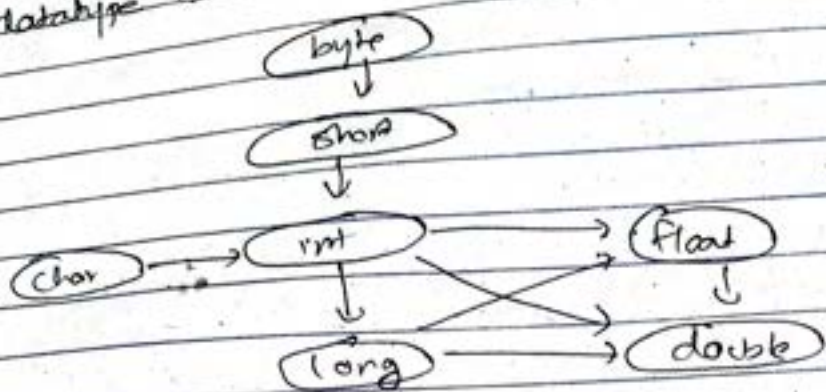
```
2 {
3     Test t = new Test();
```

```
4     t.show('a');
5 }
```

→ output
int method. reason

Automatic Promotion

→ One type is promoted to another implicitly if no matching datatype is found. Below is the diagram:



Object is parent of all class

class Test

```

{
    void show (Object a)
    {
        System.out.println ("Object method");
    }

    void show (String a)
    {
        System.out.println ("String method");
    }

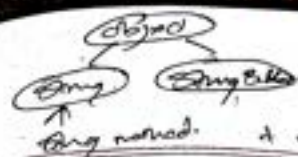
    public static void main (String[] args)
    {
        Test t = new Test();
        t.show('a');      // t.show("abc") → then String method
    }
}
  
```

Object
String

Output
- Object method.

Note:

While resolving overloaded methods, compiler will always give precedence for the child type argument.



String method: t.show("abc");

class Test

```
1 void show (StringBuffer c)
```

```
2
```

```
3 System.out.println("StringBuffer method");
```

```
void show (String a)
```

```
4
```

```
5 System.out.println("String method");
```

```
6
```

```
public static void main (String[] args)
```

```
1 Test t = new Test();
```

```
2 t.show("abc"); // String method
```

```
3 t.show(new StringBuffer("xyz")); // StringBuffer method
```

```
4 t.show(null); → error → confuse
```

```
5
```

```
6
```


class Test

{

void show(int a, float b)

{

System.out.println("int float method");

}

void show(float a, int b)

{

System.out.println("float int method");

}

public static void main(String[] args)

{

Test t = new Test();

t.show(10, 20.5f); → int float method

t.show(20.5f, 10); → float int method

t.show(10, 20); → ambiguous error

}

}

Method Overriding (Run-Time polymorphism)

Date: _____
Page: _____

class Test

// no arg

{

void show()

{ System.out.println("1");

}

}

class X42 extends Test

{

@Override

void show()

{

System.out.println("2");

}

public static void main (String[] args)

{

Test t = new Test();

t.show(); // 1

X42 x = new X42();

x.show(); // 2

}

}

// Type of arg

{
class Test

void show(String a)
{
system.out.println("1");
}

}

{
class XYZ extends Test

void show(String a)
{
system.out.println("2");
}

public static void main(String[] args)

{
Test t = new Test();
t.show("zunu"); // 1

XYZ x = new XYZ();
x.show("zunu"); // 2

}

}

class Test

// seq of args

```
{  
    void Show (String a, int b)  
    {  
        System.out.println("1");  
    }  
}
```

class X42 extends Test

```
{  
    void Show (String a, int b)  
    {  
        System.out.println("2");  
    }  
}
```

```
public static void main (String[] args)
```

```
{  
    Test t = new Test();  
    t.Show ("suru", 123);
```

```
    X42 x = new X42();  
    x.Show ("rama", 456);  
}
```

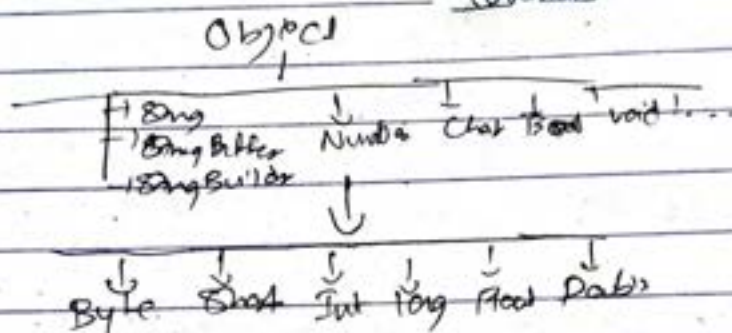
Note:

→ Method overriding allows a subclass or child class to provide a specific implementation of a method that is already provided by one of its superclasses or parent classes. The implementation in the subclass overrides (replaces) the

implementation in the superclass by providing a method that has same name, same parameters or signature, and same return type as the method in parent class.

Important Qns.

- 1) Do overriding method must have same return type/return type.
- J.4 before same return type. but after 1.4 we can use Java 5.0
 - parent class wise (child's return type should be sub type of parent's return type) covariant return type



class Test

```

1  object show()
2  {
3      Sop("1");
4  }
  
```

⚡

class X42 extends Test

```

1  {
2      String show()
3      {
4          Sop("2");
5      }
6  }
  
```

⚡

↑ Sum()

```

1  Test t = new Test();
2  t.show();
  
```

12

12

```

1  X42 ob = new X42();
2  ob.show();
  
```

2

2) Overriding and Access-Modifiers

The access modifier for an overriding method can allow more, but not less, access than the overridden method. For eg: a protected instance method in the super-class can be made public, but not private in the subclass. Doing so, will generate compile-time error.

class Test

```
{  
    [public] Object show()  
    {  
        Sop("1");  
    }  
}
```

class XYZ extends Test

```
{  
    public String show() ← greater  
    {  
        Sop("2");  
    }  
}
```

3) Overriding and Exception-Handling

→ Below are two rules to note when overriding methods related to exception handling.

Rule 1: If the super-class overridden method does not throws an exception, subclass overriding method can only throws the unchecked exception, throwing checked exception will lead to compile-time error.

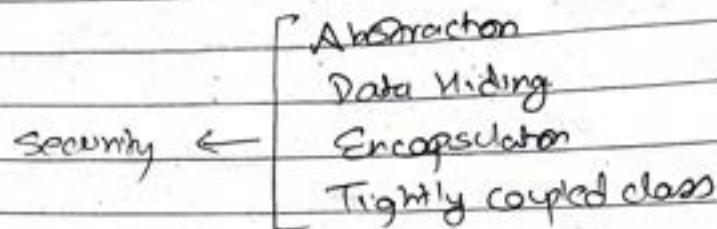
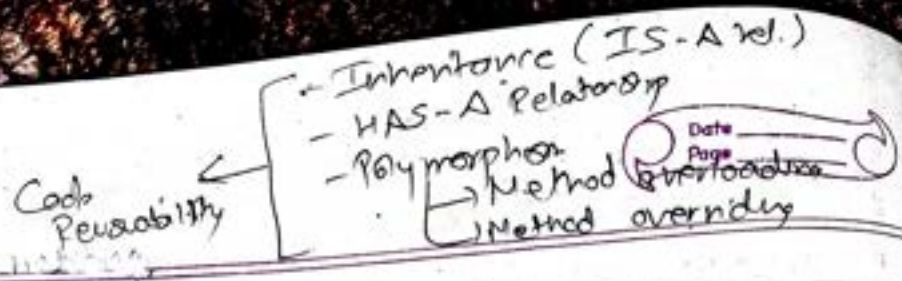
Rule 2: If the super-class overridden method does throws

Date _____
Page _____

an exception, subclass overriding method can only throw same, subclass exception. Throwing parent exception. Exception hierarchy will lead to compile-time error. Also there is no issue if subclass overriding method is not throwing any exception.

Note:

- Final methods cannot be overridden
- Static methods cannot be overridden
- Private methods cannot be overridden



Abstraction ← By using abstract class (0-100%)
By interfaces (100%)

<u>Abstraction</u>	<u>Encapsulation</u>
→ Detail Hiding	→ Data Hiding
→ Implementation Hiding	→ Information Hiding
- It deals with exposing the interface to the user & hiding the details of implementation	- Encapsulation groups together data & methods that act upon data.

car
 ↳ break
 ↳ steering
 ↳ horn
 ↳ gear

Shows to driver (Abstract) but driver doesn't need to know internal functioning of those.

A method without body (no implementation) is known as abstract method.

Cpr → class
 no. of types = 4
 1. start()
 2. sort() sort with req.

no. of types = 4

1) Start()

Scoring → class

no. of types = 2.

```

11  print()
12  sub('Bon',

```

Vehicle
↳ no. & types;
Cost;

abstract class Vehicle

```
int na. & hysres;
```

```
abstract void start();
```

3

class car extends vehicle

27

void start()

2

```
System.out.println("Starts with key");
```

2,

2

class scooter extends vehicle

2

```
void strA()
```

2

System.out.println("Starts with tick");

2

2

public; class Main {

```

public class Main {
    public static void main (String[] args)
    {
        new Car();
    }
}

```

22

```
Car c = new Car();
```

```
c. BarA();
```

```
Scooter s = new Scooter();
```

```
s = startA();
```

2.

3


```
abstract class Shape
```

```
{
```

```
    abstract void draw(); // abstract method.
```

```
}
```

```
class Circle extends Shape
```

```
{
```

```
    void draw()
```

```
{ System.out.println("Drawing circle");
```

```
}
```

```
}
```

```
class Rectangle extends Shape
```

```
{
```

```
    void draw()
```

```
{ System.out.println("Drawing rectangle");
```

```
}
```

```
}
```

```
public class Main {
```

```
    public static void main (String[] args) throws Exception
```

```
{
```

```
        Circle c = new Circle();
```

```
        c.draw();
```

```
}
```

```
}
```

Notes

1) A m

as

2) A n

class

method

3) If a

class

is

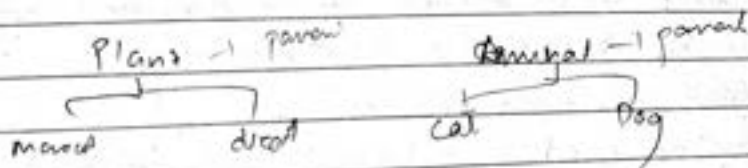
abstract

4) A

class

Notes:

- 1) A method without body (no implementation) is known as abstract method.
- 2) A method must always be declared in an abstract class, or we can say that if a class has an abstract method, it should be declared abstract as well.
- 3) If a regular class extends an abstract class, then the class must have to implement all the abstract methods of abstract parent class or it has to be declared abstract as well.
- 4) Abstract classes cannot be instantiated, means we can't create an object of abstract class.



Interface

- Interfaces are similar to Abstract class but having all the methods of abstract type.

<pre>abstract class Test { abstract void show(); void display(); }</pre>	<pre>-interface I1 { //abstract methods }</pre>
--	---

- Interfaces are the blueprints of the class.
- It specifies what a class must do and not how.
- It is used to achieve abstraction.
- It supports multiple inheritance.
- It can be used to achieve loose coupling.

Loose coupling is a design goal that seeks to reduce the inter-dependencies between components of a system with the goal of reducing the risk that changes in one component will require changes in any other component. Loose coupling is much more generic concept intended to increase the flexibility of system, making it more maintainable and makes the entire framework more stable.

interface InterfaceName;

↳ methods // abstract
public

fields // public static final

→ can create default concrete method in 8th version
→ can create static method

Interface I1

§

public abstract void show(); // abstract method

public static final int a = 10; // field

default void display()

§

§

public static void run()

§

§

§

8th version

→ default concrete method

→ static method

9th version

→ private method

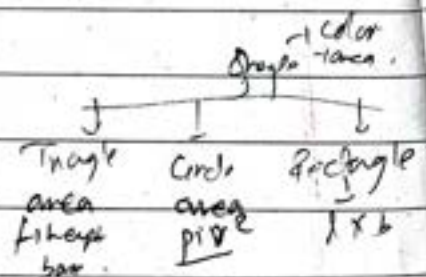
Conditions for method overriding

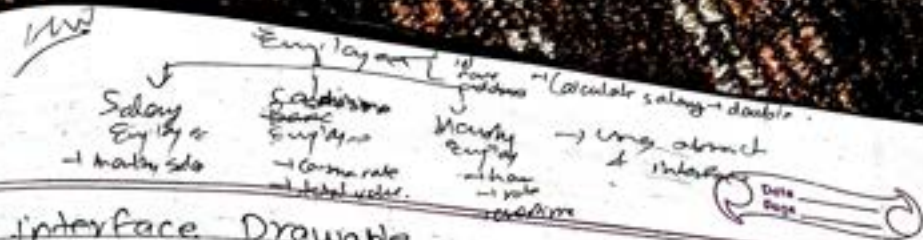
- same name
- different class
- same arguments
 - no. of arg.
 - type of arg.
 - sequence of arg.
- inheritance.

```
interface I1
{
    public void show();
}
```

```
class Test implements I1
{
    public void show()
    {
        System.out.println("1");
    }
}
```

```
public class main
{
    public static void main (String[] args)
    {
        Test t = new Test();
        t.show();
    }
}
```





interface Drawable

{ public void draw();

}

class Circle implements Drawable

{

public void draw()

{

System.out.println("Drawing circle");

}

}

class Rectangle implements Drawable

{

public void draw()

{

System.out.println("Drawing rectangle");

}

}

public class main

{ public static void main (String[] args)

{

Circle circle = new Circle();

circle.draw();

}

}

Inheritance

Date _____
Page _____

- It is inheriting the properties of parent class into child class
- Inheritance is the procedure by which one object acquires all the properties and behaviours of a parent object.

Parent class / Super class

```
class Animal
{
    void eat()
    {
        System.out.println("I am eating");
    }
}
```

Child class / Sub-class

```
class Dog extends Animal
{
    public static void main(String[] args)
    {
        Dog d = new Dog();
        d.eat();
    }
}
```

Dog IS-A Animal
↓ child ↓ parent

⇒ Sparrow IS-A Bird

⇒ Car IS-A Vehicle

⇒ Surgeon IS-A Doctor

Advantages

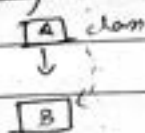
- 1) Code Reusability
- 2) We can achieve polymorphism using inheritance

Disadvantages

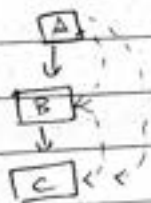
- 1) classes are tightly-coupled.

Types of Inheritance

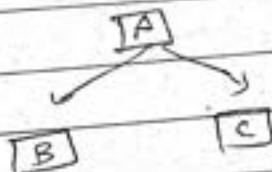
- i) Single Inheritance



- ii) Multilevel Inheritance



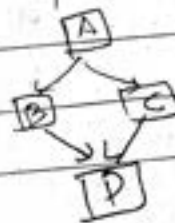
- iii) Hierarchical Inheritance



- iv) Multiple Inheritance



- v) Hybrid Inheritance



Single Inheritance

```
class A
```

```
{
```

```
    void Show A()
```

```
{
```

```
        System.out.println("a class method");
```

```
}
```

```
}
```

```
class B extends A
```

```
{
```

```
    void Show B()
```

```
{
```

```
        System.out.println("b class method");
```

```
}
```

```
}
```

```
}
```

```
public class main
```

```
{    public static void main (String [] args)
```

```
{
```

```
    A obj1 = new A();
```

```
    obj1.Show A();
```

```
    obj1.Show B();
```

```
    B obj2 = new B();
```

```
    obj2.Show A();
```

```
    obj2.Show B();
```

```
}
```

```
}
```


Multi level Inheritance

class A

void showA()

{

System.out.println("a class method");

}

}

class B extends A

void showB()

{

System.out.println("b class method");

}

}

class C extends B

{

void showC()

System.out.println("c class method");

}

}

public class main {

public static void main (String[] args)

{

A ob1 = new A();

ob1.showA();

B ob2 = new B();

ob2.showA();

ob2.showB();

C ob3 = new C();

ob3.showA();

ob3.showB();

ob3.showC();

Hierarchical



class A

void showA()

{

System.out.println("a class method");

}

}

class B extends A

void showB()

{

System.out.println("b class method");

}

}

class C extends A

{

void showC()

{

System.out.println("c class method");

}

}

public class main

public static void main(String[] args)

{

A obj1 = new A();

obj1.showA();

B obj2 = new B();

obj2.showA();

obj2.showB();

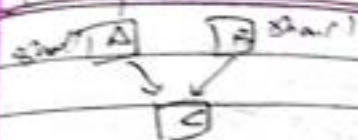
C obj3 = new C();

obj3.showA();

obj3.showC();

}

Multiple



class C extends A, B

{
 sum()

→ Ambiguity error

C ob = new C(),

ob.start()

which class A or B?

class A

{
 A()

// constructor

{
 private void start()

}

}

}

}

}

}

}

}

}

}

}

}

}

}

}

}

}

}

}

}

}

}

}

}

}

}

}

}

}

class B extends A

{

}

}

}

}

}

}

}

}

}

}

}

}

}

}

}

}

}

}

}

}

}

}

}

}

}

}

}

}

}

}

}

}

class A extends Object → default parent class

Types of Relationship between Classes

Inheritance

IS-A

- extends keyword
- blood relation
- tightly coupled

Association

HAS-A

- reference variable / handle
- non blood relation
- not tightly coupled

Aggregation
(weak bonding)

Composition
(Strong bonding)

Advantages

- 1) Code Reusability
- 2) Cost Cutting
- 3) Reduce Redundancy (unnecessary code remove)

Inheritance (IS-A)

class A

{

✓

}

class B extends A

{

→

}

class Vehicle

{

}

class car extends Vehicle

{

}

Car **ISA** Vehicle

Association (HAS-A)

Date _____
Page _____

Class Student

{

String name;

int roll-no;

}

Student HAS-A name

Student HAS-A roll-no

Class Engine

{

Class Car

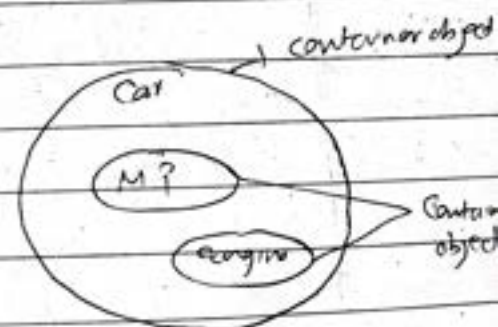
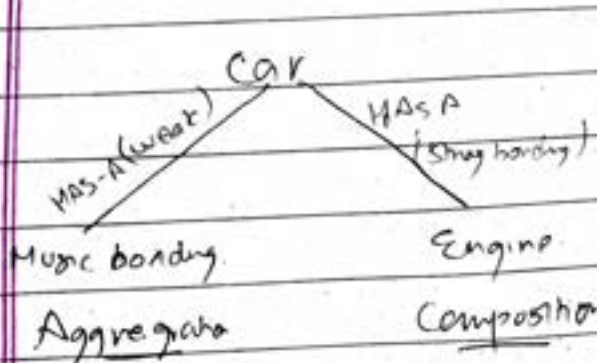
{

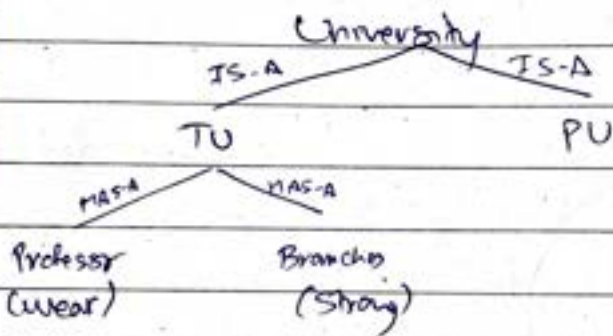
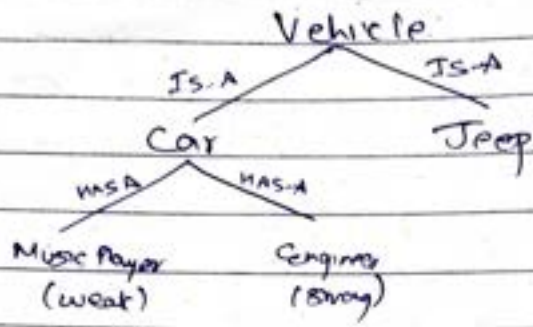
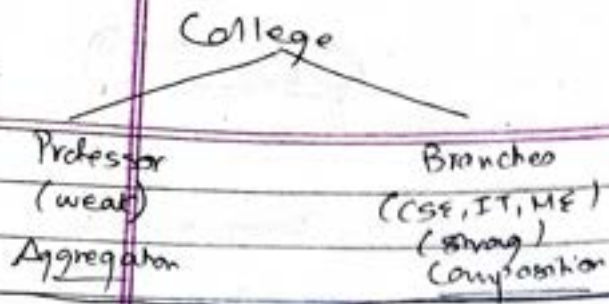
Engine e = new Engine();

}

Car HAS-A engine

→ Classes are not tightly coupled.





Date _____
Page _____

Inheritance (IS-A): Child class object comes the body of the Parent class when initiated. Moreover there are certain privileges attach to method overriding to the classes related this way. This relationship exist for code reuse, method overriding and interfacing (through abstract class). IS-A Relationship is achieved by 'extends' keyword.

Association (HAS-A): Association is relation between two separate classes which establishes through their objects. Association can be one-to-one, one-to-many, many-to-one, many-to-many.

In OOP, an object communicates to other object to use functionality and services provided by that object. There are two forms of association:

- i) Aggregation
- ii) Composition.

Aggregation: Without existing container object i.e. there is a chance of existing contained object, then container & contained objects are weakly associated & this weak association is known as aggregation.

For example: College consists of several professors, without existing college, there may be chance of existing professors; hence, college & professor objects are weakly associated & this is known as aggregation. In aggregation, container object holds just reference of contained objects.

For eg:

```
Final class College
{
    private Professor professor;
    void setProfessor (Professor professor)
    {
        this.professor = professor;
    }
    void teach()
    {
        if (professor != null)
            professor.teach();
    }
}
```

Composition: Without existing container object if there is no chance of existing contained objects, then container & contained objects are strongly associated & this strong association is composition.
For eg: College consists of several branches, without existing college, there is no chance of existing branches, hence college & branches are strongly associated & this is known as composition.

In composition container object holds directly contained objects.